# `std::integral_constant` Literals


| | |
|---|---|
| Document #: | P2725R1 |
| Date: | 2022-11-18 |
| Project: | Programming Language C++ |
| Audience: | LEWG-I<br>LEWG |


| | |
|---|---|
| Reply-to: | Zach Laine<br><whatwasthataddress@gmail.com> |


# § 1 Changelog

## § 1.1 Changes since R0

— Add missing logic for `"0B"` binary literal prefixes.

— Add a discussion of the interaction between `integral_constant::operator-` and implicit conversion to `T`.

# § 2 The ergonomics of `std::integral_constant<int>` can be improved

`std::integral_constant<int>` is used in lots of places to communicate a constant integral value to a given interface. The length of its spelling makes it very verbose. Fortunately, we can do a lot better.

**Before**

```
// From P2630R1
auto const sir =
  strided_index_range{integral_constant<size_t, 0>{},
                      integral_constant<size_t, 10>
        {},
                      3};
auto y = submdspan(x, sir);
```

**After**

```
using namespace
        std::literals::integal_constant_literals;
// strided_index_range{0ic, 10ic, 3} would work,
        too.
auto y = submdspan(x, strided_index_range{0uzic,
        10uzic, 3});
```

The use of `std::integral_constant<size_t>` above is fairly limited; there are only two specializations present. However, more complicated expressions containing more `std::integral_constant<size_t>`s benefit even more from the terseness of literals:

**Before**

```
auto const sir =
  compute_index_range(integral_constant<size_t, 0>{},
                      integral_constant<size_t, 3>{},
                      integral_constant<size_t, 8>{},
                      integral_constant<size_t, 5>{},
                      integral_constant<size_t, 2>{},
                      integral_constant<size_t, 10>
        {});
auto y = submdspan(x, sir);
```

**After**

```
using namespace
        std::literals::integal_constant_literals;
auto const sir =
  compute_index_range(0ic, 3ic, 8ic, 5ic, 2ic,
        10ic);
auto y = submdspan(x, sir);
```

The absence of syntactic noise when using literals is dramatic in this case. With the advent of `std::mdspan` and `std::submdspan()`, longer expressions involving more `std::integral_constants` are more likely to occur for more users.

# 3 These literals may open up other code-improvement opportunities

If we had easily spelled integral constants, we could add an interface to get values out of tuples using a much more natural syntax.

For instance, if we added an index operator to `std::tuple` that takes a `std::integral_constant<size_t>`:

| Before | After |
|---|---|
| `auto t = tuple<int, string>(0, "some text");`<br>`get<1>(t) = "some different text";` | `using namespace std::literals::integal_constant_literals;`<br>`auto t = tuple<int, string>(0, "some text");`<br>`t[1ic] = "some different text";` |

Or, if we added an overload of `std::get()` that takes a `std::integral_constant<size_t>`:

| Before | After |
|---|---|
| `auto t = tuple<int, string>(0, "some text");`<br>`get<1>(t) = "some different text";` | `using namespace std::literals::integal_constant_literals;`<br>`auto t = tuple<int, string>(0, "some text");`<br>`get(t, 1ic) = "some different text";` |

Modifications to `std::tuple` and/or `std::get` are out of scope for this paper, but they show the utility of `std::integral_constant` literals.

# 4 Proposed design

Add a UDL for each permutation `P` of the literal suffixes in [lex.icon]/integer-literal, with the suffix "`ic`" appended to `P` (e.g. "`ull`" + "`ic`" = "`ullic`"). Each UDL will return a `std::integral_constant<T, V>`, where `T` is the type that corresponds to `P`, and `V` is the integral value specified by the characters given to the UDL.

Note that the simpler suffix "`c`" (for "`const`") is not usable, because `c` is a valid hex digit. This makes us sad.

Each call to one of the proposed UDLs is ill-formed when the number parsed is `> std::numeric_limits<T>::max()`. This matches user expectations when writing integral literals with out-of-bounds values, except that it turns UB into ill-formed code. This is almost certainly an improvement, since it happens at compile time (UB is highly problematic at compile time).

Since a UDL will never include a sign character, in order to be able to write natural-looking literals like `-42ic`, we also need to add a unary `operator-` for `std::integral_constant`.

## 4.1 A minor limitation

You cannot spell the minimal value for signed integral types using the proposed literals, because we are not generating negative constants by parsing integers (we never see the minus sign, remember). Instead, we are parsing unsigned integers and then using unary negation. Assuming `INT_MIN == -2147483648` and `INT_MAX == 2147483647`:

```
-2147483648ic; // Ill-formed!
```

Before the negation can occur, the implementation must reject `2147483648ic`, because it is greater than `INT_MAX`. There is simply no way to spell `std::integral_constant<T, std::numeric_limits<T>::min()>` as a literal, if `T` is signed. Fortunately, one can still spell it the verbose way, as you saw one sentence ago.

## 4.2 What about `std::integral_constant<T, x>`'s implicit conversion to `T`?

You might have already noticed that this is well-formed:

```
std::integral_constant<int, 5> five;
int negative_five = -five;
assert(negative_five == -5);
```

That's because `std::integral_constant<T, x>` is implicitly convertible to a `T` whose value is `x`.

Doesn't adding an `operator-` to `std::integral_constant` change the meaning of existing code? Yes and no. In the example above, there is no user-visible change. The expression `-five` becomes `std::integral_constant<int, -5>{}` instead of `int(-5)`. The result is the same, though – the implicit conversion still occurs, but it occurs after the negation instead of before it.

There are other cases that result in a user-visible change, though:

1. `decltype(-five)` will be very different, which might break some constrained templates, for example:

```
void f(std::same_as<int>);
f(-std::integral_constant<int, 1>{}); // valid now, ill-formed with P2725
```

2. In the case of any minimum signed value (e.g. INT_MIN), `-std::integral_constant<T, std::numberic_limits<T>::min()>{}` changes from UB to ill-formed with the introduction of `std::integral_constant::operator-`.

3. is an example of change that we typically accept – it will not silently change the meaning of code, except in some squirrely overload resolution cases, and is easily if awkwardly fixed by prepending a `+` at the call site. I think that 2) is an improvement.

# § 5 Implementation experience

An `integral_constant` with the proposed semantics has been a part of Boost.Hana since its initial release in May of 2016. Its literals have been used by many, many users.

Using the Hana implementation as a guide, I independently implemented the UDLs in this paper, including checks (missing from the Hana implementation) that the parsed number can fit within the range of the integral type associated with the UDL. It was straightforward; it took me about 4 hours, including tests. The implementation can be found on Github. The C++26 implementation is nearly trivial, since `std::from_chars()` is `constexpr` in C++23 and later.

# § 6 Wording

In 21.3.3 [meta.type.synop], append to [meta.help]:

```
inline namespace literals::inline integral_constant_literals {
  // [meta.help.literals], suffix for integral_constant literals
  template <char ...Chars>
  constexpr auto operator"" ic();

  template <char ...Chars>
  constexpr auto operator"" uic();
  template <char ...Chars>
  constexpr auto operator"" Uic();

  template <char ...Chars>
  constexpr auto operator"" lic();
  template <char ...Chars>
  constexpr auto operator"" Lic();

  template <char ...Chars>
  constexpr auto operator"" luic();
  template <char ...Chars>
  constexpr auto operator"" ulic();
  template <char ...Chars>
  constexpr auto operator"" Ulic();
  template <char ...Chars>
  constexpr auto operator"" lUic();
  template <char ...Chars>
```

```
    constexpr auto operator"" Luic();
    template <char ...Chars>
    constexpr auto operator"" uLic();
    template <char ...Chars>
    constexpr auto operator"" ULic();
    template <char ...Chars>
    constexpr auto operator"" LUic();

    template <char ...Chars>
    constexpr auto operator"" llic();
    template <char ...Chars>
    constexpr auto operator"" LLic();

    template <char ...Chars>
    constexpr auto operator"" lluic();
    template <char ...Chars>
    constexpr auto operator"" ullic();
    template <char ...Chars>
    constexpr auto operator"" Ullic();
    template <char ...Chars>
    constexpr auto operator"" llUic();
    template <char ...Chars>
    constexpr auto operator"" LLuic();
    template <char ...Chars>
    constexpr auto operator"" uLLic();
    template <char ...Chars>
    constexpr auto operator"" ULLic();
    template <char ...Chars>
    constexpr auto operator"" LLUic();

    template <char ...Chars>
    constexpr auto operator"" zic();
    template <char ...Chars>
    constexpr auto operator"" Zic();

    template <char ...Chars>
    constexpr auto operator"" zuic();
    template <char ...Chars>
    constexpr auto operator"" uzic();
    template <char ...Chars>
    constexpr auto operator"" Uzic();
    template <char ...Chars>
    constexpr auto operator"" zUic();
    template <char ...Chars>
    constexpr auto operator"" Zuic();
    template <char ...Chars>
    constexpr auto operator"" uZic();
    template <char ...Chars>
    constexpr auto operator"" UZic();
    template <char ...Chars>
    constexpr auto operator"" ZUic();
  }
```

In 21.3.4 [meta.help]:

```
namespace std {
  template<class T, T v> struct integral_constant {
    static constexpr T value = v;

    using value_type = T;
    using type = integral_constant<T, v>;

    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }

    constexpr integral_constant<T, -v> operator-();
  };
}
```

Add new section [meta.help.op] to the end of 21.3.4 [meta.help]:

```
constexpr integral_constant<T, -v> operator-();
```

*Mandates:* `-v` is a value representable by `T`.

*Returns:* `{}`

Add subsequent new section [meta.help.literals] after new section [meta.help.op]:

```
namespace std::inline literals::inline integral_constant_literals {
  struct ic_base_and_offset_result // exposition only
  {
    int base;
    int offset;
  };

  template<size_t Size, char... Chars>
  constexpr ic_base_and_offset_result ic_base_and_offset() // exposition only
  {
      constexpr char arr[] = {Chars...};
      if constexpr (arr[0] == '0' && 2 < Size) {
        constexpr bool is_hex = arr[1] == 'x' || arr[1] == 'X';
        constexpr bool is_binary = arr[1] == 'b' || arr[1] == 'B';

        if constexpr (is_hex)
          return {16, 2};
        else if constexpr (is_binary)
          return {2, 2};
        else
          return {8, 1};
      }
      return {10, 0};
  }

  template<class TargetType, char ...Chars>
  constexpr TargetType ic_parse() // exposition only
  {
    constexpr auto size = sizeof...(Chars);

    constexpr auto bao = ic_base_and_offset<size, Chars...>();
    constexpr int base = bao.base;
    constexpr int offset = bao.offset;

    const auto f = std::begin(arr) + offset, l = std::end(arr);
    TargetType x{};
    constexpr auto result = from_chars(f, l, x, base);
    return result.ptr == l && result.ec == errc{} ? x : throw logic_error("");
  }
}

template <char ...Chars>
constexpr auto operator"" ic();
template <char ...Chars>
constexpr auto operator"" uic();
template <char ...Chars>
constexpr auto operator"" Uic();
template <char ...Chars>
constexpr auto operator"" lic();
template <char ...Chars>
constexpr auto operator"" Lic();
template <char ...Chars>
constexpr auto operator"" luic();
template <char ...Chars>
constexpr auto operator"" ulic();
template <char ...Chars>
constexpr auto operator"" Ulic();
template <char ...Chars>
constexpr auto operator"" lUic();
template <char ...Chars>
constexpr auto operator"" Luic();
template <char ...Chars>
```

```
constexpr auto operator"" uLic();
template <char ...Chars>
constexpr auto operator"" ULic();
template <char ...Chars>
constexpr auto operator"" LUic();
template <char ...Chars>
constexpr auto operator"" llic();
template <char ...Chars>
constexpr auto operator"" LLic();
template <char ...Chars>
constexpr auto operator"" lluic();
template <char ...Chars>
constexpr auto operator"" ullic();
template <char ...Chars>
constexpr auto operator"" Ullic();
template <char ...Chars>
constexpr auto operator"" llUic();
template <char ...Chars>
constexpr auto operator"" LLuic();
template <char ...Chars>
constexpr auto operator"" uLLic();
template <char ...Chars>
constexpr auto operator"" ULLic();
template <char ...Chars>
constexpr auto operator"" LLUic();
template <char ...Chars>
constexpr auto operator"" zic();
template <char ...Chars>
constexpr auto operator"" Zic();
template <char ...Chars>
constexpr auto operator"" zuic();
template <char ...Chars>
constexpr auto operator"" uzic();
template <char ...Chars>
constexpr auto operator"" Uzic();
template <char ...Chars>
constexpr auto operator"" zUic();
template <char ...Chars>
constexpr auto operator"" Zuic();
template <char ...Chars>
constexpr auto operator"" uZic();
template <char ...Chars>
constexpr auto operator"" UZic();
template <char ...Chars>
constexpr auto operator"" ZUic();
```

*Returns*: `integral_constant<T, v>{}`, where `T` is the integral type associated with the *integer-prefix* corresponding to the function name without the “ic” suffix, and `v` is `ic_parse<T, Chars...>()`.

Add a new feature macro, `__cpp_lib_integral_constant_literals`.

# § 7 Acknowledgments

Thanks to Matthias Kretz for his thoughtful comments on this paper.